Using a Relative Standard Deviation Fixed-Width Stopping Rule as a Markov Chain Monte Carlo Diagnostic

Olof Heurgren

Bachelor's Thesis in Computer Science
at Stockholm University, Sweden 2017

Stockholms universitet

NADA

Using a Relative Standard Deviation Fixed-Width Stopping Rule as a Markov Chain Monte Carlo Diagnostic

Olof Heurgren

Bachelor's Thesis in Computer Science (15 ECTS credits)
Single Subject Course
Stockholm University year 2017
Supervisor at NADA was Lars Arvestad
Examiner was Jens Lagergren

Department of Numerical Analysis and Computer Science
Royal Institute of Technology, School of Computer Science and Communication
SE-100 44 Stockholm, Sweden

Abstract

In this project a stopping rule ("FWSR"), designed for deciding when to terminate a running Markov Chain Monte Carlo (MCMC) simulation, is implemented and evaluated as a run length diagnostic in a piece of MCMC post-processing software called VMCMC. It is concluded that FWSR has relatively high demands in terms of run length and that it seems to favour good mixing. Discarding burn-in does not appear to have a significant effect on FWSR results.

Sammanfattning

En stoppregel som efterbehandlingsdiagnostik för Markov Chain Monte Carlo

I detta projekt implementeras och utvärderas en stoppregel (här kallad "FWSR") för Markov Chain Monte Carlo (MCMC), avsedd för att avgöra när en pågående simuering ska avbrytas, som en efterbehandlingsdiagnostik i ett program kallat VMCMC. Det konstateras att FWSR verkar ha relativt höga krav på körlängd av en simulering och att metoden verkar föredra god mixning av MCMC-utdata. Att kasta bort burn-in delen av utdatat verkar dock ha en försumbar effekt på FWSR:s utslag.

Acknowledgements

I would like to thank the supervisor of this project Lars Arvestad, Senior Lecturer at Stockholm University, for providing the thesis subject along with valuable assistance and feedback during the project. I would also like to thank Prof. Jens Lagergren, at KTH Royal Institute of Technology, for examining the report. Lastly I would like to thank Caroline Nordquist, educational counsellor at the Department of Numerical Analysis and Computer Science (Nada) at Stockholm University, in part for examining the report but above all for always being of such great help to me throughout my studies at Nada.

Contents

1 Introduction **1**

2 Problem statement **2**

3 Theory **3**
3.1 Phylogenetics 3
3.2 Markov Chain Monte Carlo (MCMC) 3
3.3 MCMC convergence 4
3.3.1 Burn-in and convergence in distribution 4
3.3.2 Run length 5
3.3.3 Connection between burn-in and run length 5
3.3.4 Mixing 5
3.4 VMCMC 5
3.5 A Relative Standard Deviation Fixed-Width Stopping Rule (FWSR) 6
3.5.1 Theory 6
3.5.2 Summary and algorithm 8
3.6 Effective Sample Size (ESS) 8
3.7 Connection between FWSR and ESS 9
3.8 Recommended FWSR and ESS values 9

4 Analysis **10**
4.1 FWSR in a post-processing setting 10
4.2 Relevant input and output data 10

5 Results **12**
5.1 Implementation 12
5.1.1 FWSR class 12
5.1.2 Sample mean and standard deviation class 12
5.1.3 Primary command line tool 13
5.1.4 Full parameter data and plotting tool 13
5.2 Evaluation 13
5.2.1 Toy example 13
5.2.2 DS1 traces 14
5.2.3 Yeast trace 16
5.2.4 Decreasing DS1 run length 17
5.2.5 On burn-in 18
5.2.6 Note on thinning 19

6 Conclusions **20**

7 Recommendations for further study **21**

References **22**

1 Introduction

Visual MCMC (VMCMC) [1, 2, 13] is a piece of software comprising a number of tools intended to facilitate post-processing of trace data generated by Markov chain Monte Carlo (MCMC) software, specifically within the area of phylogenetics. One such tool comprises various automated diagnostics used for estimating burn-in and convergence of the trace. The authors [2] suggest implementing additional functionality in the form of a relative standard deviation fixed-width stopping rule (henceforth "FWSR") introduced by Gong and Flegal [7]. Being a stopping rule, FWSR is intended to be used during an MCMC simulation to determine when to terminate it. This text instead explores the usage of FWSR in the post-processing setting relevant to VMCMC.

2 Problem statement

The goal of this project is twofold.

1. Implement FWSR in VMCMC in a way that is relevant to its post-processing setting.
2. Investigate how FWSR behaves in this setting and how it relates to some of the diagnostics already implemented in VMCMC.

3 Theory

This section reviews theoretical concepts relevant for this essay. For the reader already familiar with MCMC and phylogenetics, the main topics of interest are those concerning FWSR and ESS. That is, parts 3.5, 3.6 and 3.7.

3.1 Phylogenetics

Phylogenetics is the study of evolutionary relationships between different kinds of life forms, that is, how different life forms are biologically related in terms of ancestry. It often entails constructing evolutionary trees that describe these relationships [12].

MCMC is used in phylogenetics both with trees as discrete parameters and with continuous numeric parameters related to the trees, for example duplication and loss rates [2], to estimate the most likely values of those parameters.

3.2 Markov Chain Monte Carlo (MCMC)

As mentioned above, in phylogenetics MCMC algorithms are used with both numbers and trees as parameters. FWSR concerns only the former of the two. Thus, MCMC methods concerning the latter will not be described here. See e.g. Sahlin [12] for a description of MCMC in relation to phylogenetic trees. Specific MCMC algorithms will not be described either, as we are primarily interested in the output of MCMC algorithms and not in how those algorithms work.

MCMC is a class of methods in Bayesian statistics for gathering (one- or multidimensional) samples from a posterior probability distribution where direct sampling is difficult. Given a random starting sample point, an MCMC algorithm iteratively generates a large sample, called the *trace* in the form of a sequence of sample points, by means of constructing an underlying *Markov Chain*. Any two successive points are usually strongly dependent [12].

After enough iterations the distribution of the trace will start to approximate the target probability distribution, with approximations improving as the number of iterations increase.

Below follows a summary of a more technical description by Gong and Flegal [7], introducing terminology and concepts upon which the description of FWSR (section 3.5) is based. Some details are omitted and can be found in the original text.

Suppose that, given a probability distribution $\boldsymbol{\pi}$ with support $\mathbf{X} \subseteq \mathbb{R}^d$, $d \geq 1$, we want to calculate the expected values of a number of $\boldsymbol{\pi}$-integrable functions $\mathrm{g}_i : \mathbf{X} \to \mathbb{R}$, $i = 1, ..., p$ for some p. The expected value of such a function is defined as

$$\mathbb{E}_\pi \mathrm{g}_i := \int_{\mathbf{X}} \mathrm{g}_i(x) \boldsymbol{\pi}(dx).$$

We define $\mathbf{g}(x) := (\mathrm{g}_1(x), ..., \mathrm{g}_p(x))^T$. Our goal then, is to calculate $\mathbb{E}_\pi \mathbf{g} = (\mathbb{E}_\pi \mathrm{g}_1, ..., \mathbb{E}_\pi \mathrm{g}_p)^T \in \mathbb{R}^p$. Doing so analytically is rarely possible in practical settings. Thus, we attempt to estimate using MCMC, which entails the construction of a Markov chain $X = \{X^{(0)}, X^{(1)}, ...\}$ ($X^{(j)} \in \mathbf{X}$, $j = 0, 1, ...$), fulfilling certain conditions (see Gong and Flegal [7]).

We can then use the sample mean as an estimator of $\mathbb{E}_\pi \mathbf{g}$ after n iterations:

$$\bar{\mathbf{g}}(n) := \frac{1}{n} \sum_{j=0}^{n-1} \mathbf{g}(X^{(j)}) = (\bar{\mathrm{g}}_1(n), ..., \bar{\mathrm{g}}_p(n))^T$$

Note that the *trace* in this terminology would be the sequence ($\mathbf{g}(X^{(0)})$, $\mathbf{g}(X^{(1)})$,...).

3.3 MCMC convergence

Typically, MCMC convergence refers to one out of the two concepts described below, where FWSR concerns the latter.

3.3.1 Burn-in and convergence in distribution

The burn-in of a trace is a beginning portion of the trace of which the end is the last point that is dependent on the starting point, meaning that the samples beyond that point accurately represent the target posterior distribution. Thus, the burn-in portion of the trace is not considered a good representation of the target distribution. We say that the remainder of the chain has converged in distribution or reached its stationary phase, meaning that we are sampling from the actual target distribution [12].

In practice it is not possible to analytically determine convergence in distribution. Instead, burn-in and convergence are typically heuristically estimated through some diagnostic, such as those included in VMCMC [2].

3.3.2 Run length

We say that we have convergence in terms of run length when the simulation has been run long enough that the estimate of the sample mean is accurate enough according to some measure [12].

3.3.3 Connection between burn-in and run length

Since the burn-in portion of the trace provides sample points that are unrepresentative of the target distribution, not discarding it will typically result in a higher sample standard deviation and the need to run the simulation longer to outweigh the unrepresentative values. Thus, run length thresholds will likely be reached sooner if the burn-in portion is removed. Running times are often high for MCMC simulations, so being able to reduce the required run length is desirable [12].

3.3.4 Mixing

A trace that has converged in distribution will have good *mixing*. That is, it does not get stuck in local optima but instead contains points evenly distributed around the mean. Assessment of good mixing can often be done by visual inspection of the trace plot. The trace should look like a caterpillar with a horizontal mean [9]. Ali et al. [2] point out good mixing both as an important aspect of a successful MCMC run and as something that can be difficult to achieve within phylogenetics.

3.4 VMCMC

VMCMC, as described by its authors Ali et al.[2], is a piece of software to be used for post-processing of MCMC trace data through both graphical and command-line interfaces. It is specifically intended for phylogeny, with support for analysis of tree parameters as well as numeric parameters. It has features such as graphical trace exploration, tree analysis tools and automated convergence and burn-in estimation for numeric parameters through various diagnostics. The last of the three mentioned features is of the most relevance to this essay. The included convergence/burn-in diagnostics are Geweke's convergence diagnostic [6], Gelman-Rubin's convergence diagnostic [5] and Höhna/Sahlin's ESS based burn-in estimator [9, 12]. VMCMC is written in Java.

The command-line interface is what will be of primary concern for the purpose of this essay. It allows for the user to get output from the various

tools on JSON [11] format, specifying desired tools and input parameters using flags.

3.5 A Relative Standard Deviation Fixed-Width Stopping Rule (FWSR)

This section describes the stopping rule introduced by Gong and Flegal [7]. The authors use "FWSR" as an abbreviation of "fixed-width stopping rule". In this essay it will instead be used as a name for the specific relative standard deviation fixed-width stopping rule that they describe. The description is based upon the terms and syntax introduced in section 3.2. As was mentioned in section 1, FWSR is intended to be used during an MCMC simulation. Section 4 introduces and discusses FWSR usage within the post-processing setting relevant to VMCMC.

3.5.1 Theory

The idea of FWSR is to terminate the simulation when an estimator's computational uncertainty is small relative to its posterior uncertainty.

As described in section 3.2, we have a chain $X = \{X^{(0)}, X^{(1)}, ...\}$ and an estimator of $\mathbb{E}_\pi \mathbf{g}$ defined as

$$\bar{\mathbf{g}}(n) := \frac{1}{n} \sum_{j=0}^{n-1} \mathbf{g}(X^{(j)}) = (\bar{\mathrm{g}}_1(n), ..., \bar{\mathrm{g}}_p(n))^T$$

The inaccuracy of $\bar{\mathbf{g}}(n)$ is denoted by the unknown Monte Carlo error $\bar{\mathbf{g}}(n) - \mathbb{E}_\pi \mathbf{g}$ [4]. Under certain regularity conditions (see e.g. Jones [10]), the Central Limit Theorem (CLT) holds for $\bar{\mathbf{g}}$. That is, for $i = 1, ..., p$:

$$\sqrt{n}(\bar{\mathrm{g}}_i(n) - \mathbb{E}_\pi \mathrm{g}_i) \xrightarrow{d} \mathrm{N}(0, \sigma_i^2)$$

as $n \to \infty$ where $\sigma_i^2 \in (0, \infty)$, meaning that the left expression converges in distribution towards a normal distribution centered at 0 with standard deviation σ_i.

If $\hat{\sigma}_i(n)$ is an estimator of σ_i, then the CLT allows for the construction of p $(1 - \delta)100$ % confidence intervals that, for $i = 1, ..., p$, have widths

$$w_i(n, \delta) = 2 z_{\delta/2} \frac{\hat{\sigma}_i(n)}{\sqrt{n}}$$

where $z_{\delta/2}$ is a critical value from the standard Normal distribution (e.g. a 95 % interval has $\delta = 0.05$ and $z_{\delta/2} = z_{0.05/2} \approx 1.96$).

The simplest fixed-width stopping rule would terminate the simulation at the iteration n for which all $w_i(n)$ are smaller than some predecided value. However, the difficulty in determining such a value beforehand in a high-dimensional setting together with the unlikelihood of finding a threshold suitable for all parameters calls for some other criterion that is easier to use. Let λ_i^2 be the posterior variance associated with $\mathbb{E}_\pi \mathrm{g}_i$, and let $\hat{\lambda}_i^2$ be an estimator of λ_i^2, defining

$$(\hat{\lambda}_1^2(n), ..., \hat{\lambda}_p^2(n))^T := \frac{1}{n-1} \sum_{j=0}^{n-1} (\mathbf{g}(X^{(j)}) - \bar{\mathbf{g}}(n))^2.$$

Let $\epsilon > 0$ be some predetermined threshold reflecting the desired accuracy of the simulation, with smaller ϵ signifying higher accuracy. FWSR terminates the simulation when the widths of all the confidence intervals are smaller than an ϵth fraction of their posterior standard deviations, that is $w_i < \epsilon \hat{\lambda}_i$ for all $i = 1, ..., p$. To avoid early termination caused by poor estimates of the σ_i^2s we add a term $p(n)$ to w_i, where a reasonable default is $p(n) = \epsilon I(n \leq n^*) + n^{-1}$, where n^* is the minimum simulation effort (the first iteration for which the stopping criterion is checked). Formally, then, the time T at which the simulation is terminated is defined by:

$$T(\epsilon, \delta) = \sup_{i \in \{1,...,p\}} \inf\{n \geq 0 : w_i(n, \delta) + p(n) \leq \epsilon \hat{\lambda}_i(n)\}$$

Gong and Flegal discuss two different ways to estimate σ_i^2 by using batch means, named "consistent batch means" (CBM) and "low-cost batch means" (LCBM) respectively. While both methods provide strongly consistent estimators, LCBM is described as being superior for a couple of reasons; the primary one being that LCBM, unlike CBM, does not require storing the entire chain in memory. However VMCMC always stores the entire chain in memory [13], and CBM is somewhat simpler to implement. So for this project, CBM is the chosen method.

When calculating batch means after n iterations, the chain is broken into a_n batches of size b_n. The mean of the jth ($j = 1, ..., a_n$) batch is defined as

$$\mathbf{Y}_j = \frac{1}{b_n} \sum_{k=(j-1)b_n}^{jb_n - 1} \mathbf{g}(X^{(k)})$$

Then, the batch means estimate of the σ_i^2s is

$$(\hat{\sigma}_1^2(n), ..., \hat{\sigma}_p^2(n))^T = \frac{b_n}{a_n - 1} \sum_{j=1}^{a_n} (\mathbf{Y_j} - \bar{\mathbf{g}}(n))^2.$$

In CBM we increase a_n and b_n as n grows, setting $b_n = \lfloor n^\tau \rfloor$ and $a_n = \lfloor n/b_n \rfloor$. Gong and Flegal recommend choosing $\tau = 1/2$ as a default value.

3.5.2 Summary and algorithm

In summary, we have the following formulas:

$$\bar{\mathbf{g}}(n) := \frac{1}{n}\sum_{j=0}^{n-1}\mathbf{g}(X^{(j)}) = (\bar{\mathrm{g}}_1(n), ..., \bar{\mathrm{g}}_p(n))^T \tag{3.1}$$

$$(\hat{\lambda}_1^2(n), ..., \hat{\lambda}_p^2(n))^T := \frac{1}{n-1}\sum_{j=0}^{n-1}(\mathbf{g}(X^{(j)}) - \bar{\mathbf{g}}(n))^2 \tag{3.2}$$

$$b_n := \lfloor n^{1/2} \rfloor \tag{3.3}$$

$$a_n := \lfloor n/b_n \rfloor \tag{3.4}$$

$$\mathbf{Y}_j := \frac{1}{b_n}\sum_{k=(j-1)b_n}^{jb_n-1}\mathbf{g}(X^{(k)}) \tag{3.5}$$

$$(\hat{\sigma}_1^2(n), ..., \hat{\sigma}_p^2(n))^T := \frac{b_n}{a_n-1}\sum_{j=1}^{a_n}(\mathbf{Y}_j - \bar{\mathbf{g}}(n))^2 \tag{3.6}$$

$$w_i(n,\delta) := 2z_{\delta/2}\frac{\hat{\sigma}_i(n)}{\sqrt{n}} \tag{3.7}$$

$$p(n) := \epsilon I(n \leq n^*) + n^{-1} \tag{3.8}$$

$$\forall i \in \{1, ..., p\} : w_i(n,\delta) + p(n) \leq \epsilon\hat{\lambda}_i(n) \tag{3.9}$$

The algorithm, then is:

Algorithm 1 FWSR

Require: $0 < \epsilon, \delta < 1$ and $n^*, m > 0$ ▷ m: iterations between checks

```
UPDATE the chain by n* iterations
while True do
    CALCULATE σ̂_i and λ̂_i for i = 1, ..., p    ▷ Equations 3.6 and 3.2
    if stopping criterion is met then           ▷ Equation 3.9
        break
    UPDATE the chain by m iterations
```

3.6 Effective Sample Size (ESS)

Since successive samples in MCMC are correlated, a chain of MCMC obtained samples for some parameter will likely hold less information about that parameter than if they had been independent. ESS is a quantity used to reflect this "information size" [12]. It measures the size of an independent and identically distributed (i.i.d) sample that has the same standard error as the MCMC chain [7]. ESS is not uniquely defined, however [7]. The approach used by Gong and Flegal [7] is defining ESS after

n iterations for $i = 1, ..., p$ as follows:

$$\mathrm{ESS}_i(n) := \frac{n}{\sigma_i^2/\lambda_i^2}$$

which they in practice estimate by

$$\widehat{\mathrm{ESS}}_i(n) := \frac{n}{\hat{\sigma}_i^2(n)/\hat{\lambda}_i^2(n)} \tag{3.10}$$

for the σ_i^2, λ_i^2, $\hat{\sigma}_i^2$ and $\hat{\lambda}_i^2$ introduced in section 3.5.1.

Gong and Flegal discuss using ESS as a stopping rule by terminating the simulation when the estimated value is sufficiently large for all parameters. Given some threshold K, we define the time $\tilde{T}$ at which the simulation terminates by

$$\tilde{T}(K) := \sup_{i \in \{1,\ldots,p\}} \inf\{n \geq 0 : \widehat{\mathrm{ESS}}_i(n) \geq K\}$$

3.7 Connection between FWSR and ESS

Gong and Flegal [7] show that FWSR and the ESS stopping rule are equivalent. They conclude that, if $T(\epsilon, \delta)$ and $\tilde{T}(K)$ are the times at which the respective rules terminate a simulation, then setting $K = 4z_{\delta/2}^2/\epsilon^2$ gives $T(\epsilon, \delta) \approx \tilde{T}(K)$.

3.8 Recommended FWSR and ESS values

Gong and Flegal [7] test the ESS stopping criterion with three thresholds; 1,000, 2,000 and 4,000, corresponding to ϵ thresholds of about 0.123, 0.088 and 0.062 respectively with $\delta = 0.05$, giving $z_{\delta/2} \approx 1.96$ (more precisely, the ESS values corresponding to those ϵ values are about 1,016, 1,984 and 3,998 according to section 3.7). In the end, they suggest $\epsilon = 0.062$ and $\delta = 0.05$ for high-dimensional settings. They note, however, that the value of ϵ has a significant effect on the number of iterations required, suggesting that it can be set higher to decrease simulation effort at the cost of accuracy.

4 Analysis

This section uses terminology and concepts introduced in section 3 and contains a discussion on various choices in implementing FWSR in VMCMC in a suitable way.

4.1 FWSR in a post-processing setting

Since VMCMC is a post-processing tool FWSR will not be used in the way Gong and Flegal intended, which is as a stopping rule for a running simulation. Rather, it will have to be used as a run length diagnostic. For this purpose, there appears to be two main uses of FWSR; (1) showing if a simulation was run long enough given some user-specified threshold, along with the number of iterations required to reach that threshold, and (2) presenting useful numeric information, produced by FWSR, that indicates the quality of the simulation results. For (1) we can run the FWSR algorithm (section 3.5.2) either until the stopping criterion is successfully met, or until the final iteration is reached without success. For (2) we can calculate relevant values, e.g. confidence interval widths or standard deviations, at the final iteration to get an indication of what threshold values would have been enough in (1).

4.2 Relevant input and output data

This section discusses which parameters may be relevant as input or output for FWSR in VMCMC.

As described in section 3.5, FWSR terminates the simulation at

$$T(\epsilon, \delta) = \sup_{i \in \{1,\ldots,p\}} \inf\{n \geq 0 : w_i(n, \delta) + p(n) \leq \epsilon \hat{\lambda}_i(n)\}$$

For $\hat{\lambda}_i(n) \neq 0$, we can write

$$w_i(n, \delta) + p(n) \leq \epsilon \hat{\lambda}_i(n) \iff \frac{w_i(n, \delta) + p(n)}{\hat{\lambda}_i(n)} \leq \epsilon$$

That fraction should then be a measurement of the quality of the trace data in terms of run length. Thus, we introduce

$$\varepsilon_i(n) := \frac{w_i(n, \delta) + p(n)}{\hat{\lambda}_i(n)} \tag{4.1}$$

as an output value of interest, where trace quality improves as $\varepsilon_i \to 0$.

The shown connection between FWSR and ESS, that setting $K = 4z^2_{\delta/2}/\epsilon^2$ gives $T(\epsilon, \delta) \approx \tilde{T}(K)$, allows us to formulate ϵ in terms of K and δ, namely

$$\epsilon = \sqrt{4\frac{z^2_{\delta/2}}{K}}.$$

Thus we can use ϵ and δ as input parameters, *or* we can equivalently use a user-specified ESS threshold with some predefined δ.

For showing if a simulation was run long enough, appropriate output data seem to be "success"/"failure" as well as the number of iterations needed to reach the desired accuracy if the simulation was successful.

For indication of trace quality we will use estimated ESS value and the aforementioned ε_i for each parameter at the last iteration. Those two values should be easy to understand and use intuitively even for those who do not know exactly how FWSR works. For users who *do* know how it works, computed confidence interval widths $w_i(n, d)$ and standard deviations $\hat{\lambda}_i(n)$ at the last iteration may also be of interest.

Since burn-in estimation is an important aspect of VMCMC, it should also be possible to specify a burn-in value for discarding an initial portion of the chain when running FWSR.

It might also be useful to have support for plotting FWSR output per iteration for a parameter. It is beyond the time scope for this project to integrate such a feature with the graphical interface of VMCMC. However, functionality to receive per-iteration FWSR data for a parameter should be implemented.

5 Results

5.1 Implementation

This section describes the implementation of FWSR in VMCMC. It does not contain code or implementation details, but gives a general account of functionality and interface. The full source code is available online [14]. The entire implementation is written in Java, with the exception of the plotting tool mentioned in section 5.1.4 which is written in Python 3.

5.1.1 FWSR class

The FWSR class[1] was implemented to easily allow extensions of FWSR functionality, with member functions to poll for various values relating to FWSR and to tweak different parameters. A class instance is constructed with a `List` of `Lists` of Doubles representing the trace for all numeric parameters.

5.1.2 Sample mean and standard deviation class

The calculation of sample mean, $\bar{g}_i$, and standard deviation, $\hat{\lambda}_i$ (equations 3.1 and 3.2), is done through the algorithm described in Biesel [3]. The algorithm allows updating sample mean and standard deviation when samples are added without recalculating for the previous samples. Gong and Flegal [7] recommend using it for LCBM (section 3.5.1) as updating $\bar{g}_i$ and $\hat{\lambda}_i$ without the need for recalculation is necessary for that approach. However, it is useful even when not using LCBM since it is faster not having to recalculate $\bar{g}_i(n-1)$ and $\hat{\lambda}_i(n-1)$ when calculating $\bar{g}_i(n)$ and $\hat{\lambda}_i(n)$.

Since being able to efficiently calculate mean and standard deviation of samples could be useful for features that may be added to VMCMC in the future, the algorithm was given its own class[2]. An instance of the class is constructed with a `List` L of `Double`s, representing the sample/trace for one parameter, that can be polled for mean and standard deviation for the sub list $[L[0], L[1], ..., L[n-1]]$ for some given n, $0 < n \leq length(L)$. If the

[1] See `src/main/java/se/cbb/vmcmc/libs/FWSR.java` in the source code [14].

[2] See `src/main/java/se/cbb/vmcmc/libs/SampleMeanAndStandardDeviationCalculator.java` in the source code [14].

next value polled for is larger than n, then mean and standard deviation up until n does not need to be recalculated.

5.1.3 Primary command line tool

The primary FWSR tool is run by using the flag `--fwsr` when running VMCMC from a terminal. Possible extra parameters are ϵ threshold (default 0.062), ESS threshold (default 3,998) and burn-in (default 0).

Output is given on stdout in JSON format, specifying input parameters in the form of file name, total amount of iterations, ϵ, δ, ESS and burn-in. The first output values given are result and success iteration indicating if and when FWSR succeeded. Then the following per-parameter values are given: Result (Success/Failure), success iteration, ε at the last iteration, ESS at last iteration, w at last iteration and $\hat{\lambda}$ at last iteration.

5.1.4 Full parameter data and plotting tool

Using the flag `--fwsr_full_param` *<parameter name>* *<file name>* will output text on JSON format containing per-iteration data of ε_i, $\widehat{\mathrm{ESS}}_i$, w_i and $\hat{\lambda}_i$ (equations 4.1, 3.10, 3.7 and 3.2) for the parameter i, signified by its name in the given file, for every iteration n. This is useful for plotting per-iteration FWSR output for a parameter. As mentioned earlier, FWSR integration within the graphical interface of VMCMC is beyond the scope of this project. However, a small Python 3 program[3] for plotting the output of `--fwsr_full_param` was added. The program reads the JSON output on stdin and plots the data.

5.2 Evaluation

This section provides an initial review FWSR behaviour through a number of case studies. The tested FWSR thresholds are those considered by Gong and Flegal, namely $\epsilon \in \{0.123, 0.088, 0.062\}$ with $\delta = 0.05$ corresponding to ESS values of about 1,016, 1,984 and 3,998.

5.2.1 Toy example

FWSR was tested on a sample of 10^5 i.i.d. random floating point numbers taken from the standard normal distribution, that is, the normal distribution with mean 0 and variance 1 ($N(0, 1)$).

[3]See `src/main/python/fwsrplot.py` in the source code [14].

Since the sample points were chosen independently rather than through MCMC, we would expect a computed sample standard deviation, $\hat{\lambda}$, close to 1 and an estimated ESS close to the sample size. The results were as expected, with $\hat{\lambda}(10^5) \approx 1.002$, $\widehat{ESS}(10^5) \approx 98,136$ and $\varepsilon(10^5) \approx 0.013$. Given thresholds of $\epsilon = 0.062$ and $\delta = 0.05$, corresponding to an ESS value of about 3998, FWSR determined that the simulation could have been terminated at $n = 3,721$. This is a slight underestimation as an ESS of $3,998$ would need at least $3,998$ iterations. Nevertheless, it is close to what we would expect. It might be a good idea to set the starting iteration for FWSR, n^*, to at least the ESS threshold to avoid this case of underestimation.

5.2.2 DS1 traces

The two trace files for this test were obtained by Ali et al.[2] to be used for an example application of VMCMC. They obtained the traces by running the MCMC software JPrIME twice with the same settings (the JC69 model and default parameters) on a ribosomal RNA data set, named "DS1", collected by Hedges et al.[8]. Both simulations were run for ten million iterations, with sampling every 100 iterations, resulting in two traces of size 10^5 with seven real-valued parameters and one tree parameter. Here they will be referred to as $trace1$ and $trace2$ respectively (to acquire the files, please refer to Ali et al.[2]).

Trace 1 successfully met the criterion of $\epsilon = 0.062$ (ESS $\approx 3,998$) when including 62,280 sample points, with values at the the last iteration ranging from $\varepsilon(10^5) \approx 0.05$ and $\widehat{ESS}(10^5) \approx 6,122$ to $\varepsilon(10^5) \approx 0.032$ and $\widehat{ESS}(10^5) \approx$ 16,653 for different parameters.

Trace 2, however, did not due to two parameters failing. We will look at one of those parameters: *DLRModelDensity*. In trace 1 the parameter had $\varepsilon(10^5) \approx 0.0495$ and $\widehat{ESS}(10^5) \approx 6,286$. In trace 2 it had $\varepsilon(10^5) \approx 0.0889$ and $\widehat{ESS}(10^5) \approx 1,943$, that is an ε about twice as large and an $\widehat{ESS}$ about three times smaller. Surprisingly the parameter's standard deviation, $\hat{\lambda}$, was slightly smaller in trace 2 than in trace 1; with trace 1 having $\hat{\lambda}(10^5) \approx 5.1061$ and trace 2 having $\hat{\lambda}(10^5) \approx 5.0378$. However, the parameter in trace 1 had a about half the confidence interval width, $w(10^5, \delta) \approx 0.2525$ against $w(10^5, \delta) \approx 0.4480$ in trace 2, indicating a larger $\hat{\sigma}(10^5)$ in trace 2.

We recall that $\hat{\sigma}(n)$, introduced in section 3.5.1, is an estimator of σ; the standard deviation of the normal distribution $N(0, \sigma^2)$ that the expression $\sqrt{n}(\bar{\mathrm{g}}_i(n) - \mathbb{E}_\pi \mathrm{g}_i)$ converges in distribution towards an $n \to \infty$, where $\bar{\mathrm{g}}_i(n) - \mathbb{E}_\pi \mathrm{g}_i$ is the Monte Carlo error. $\hat{\lambda}_i^2(n)$ and $\hat{\sigma}_i^2(n)$, for a parameter i,

are calculated in a similar manner. $\hat{\lambda}_i^2(n)$ is simply the sample variance:

$$\hat{\lambda}_i^2(n) = \frac{1}{n-1}\sum_{j=0}^{n-1}(\mathrm{g}_i(X^{(j)}) - \bar{\mathrm{g}}_i(n))^2$$

where $\bar{\mathrm{g}}_i(n)$ is the sample mean. $\hat{\sigma}_i^2(n)$ is instead calculated through *batch means*, $Y_{i,j}$, e.g. the means of a_n *batches* of size b_n of sample points.

$$\hat{\sigma}_i^2(n) = \frac{b_n}{a_n - 1}\sum_{j=1}^{a_n}(Y_{i,j} - \bar{\mathrm{g}}_i(n))^2$$

So if we intuitively think of $\hat{\lambda}_i$ as a measurement of the average distance to the mean, then $\hat{\sigma}_i$ is a measurement of the average distance of *batch means* to the mean. We can obtain a large $\hat{\lambda}_i$ even with sample points evenly distributed around the true mean. However, a large $\hat{\sigma}_i$ would imply that the mean varies across the trace, causing batch means to deviate from the true mean. A high $\hat{\sigma}_i$, then, is likely an implication of bad *mixing* (section 3.3.4) as the trace plot will not not have the desired smooth caterpillar shape, but rather have anomalies affecting the means of the batches containing them. An inspection of the VMCMC generated trace plots, seen in figures 5.1 and 5.2, for *DLRModelDensity* in trace 1 and trace 2 confirms this suspicion.

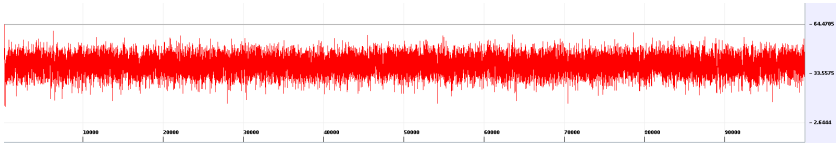

Figure 5.1: Full trace plot for the parameter *DLRModelDensity* in trace 1. 10^7 iterations with sampling every 100th iteration. The trace has a smooth caterpillar shape and a good FWSR score. x-axis: number of iterations $\times$ 100. y-axis: parameter value.

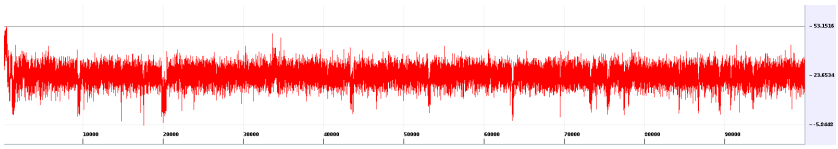

Figure 5.2: Full trace plot for the parameter *DLRModelDensity* in trace 2. 10^7 iterations with sampling every 100th iteration. The trace has a caterpillar shape, but with visible "spikes" and a much worse FWSR score than in trace 1 (figure 5.1). x-axis: number of iterations $\times$ 100. y-axis: parameter value.

Likely, the batches in trace 2 (figure 5.2) containing the "spikes" will vary in mean value from the other batches. So even though both traces have similar standard deviations, trace 2 has a higher $\hat{\sigma}$, probably caused by the anomalies.

5.2.3 Yeast trace

The trace file treated in this section was provided by the supervisor of this project, Lars Arvestad. It contains output of an MCMC simulation run on a yeast orthogroup dataset with the DLRS model (formerly "GSR", see Åkerborg et al. [15] for description of method and data), with 36 real-valued parameters and a tree parameter. The simulation was run for one million iterations with sampling every 100th iteration, yielding a trace of size 10^4; ten times smaller than the DS1 traces.

The trace was failed by FWSR even for the lowest threshold of $\epsilon = 0.123$, ESS ≈ 1016 with some parameters scoring $\widehat{ESS}(10^5)$ values just above 100 and $\varepsilon(10^5)$ three times as high as the lowest threshold. Some parameters got better scores, the best being $\varepsilon(10^5) \approx 0.0654$ with $\widehat{ESS}(10^5) \approx 3,962$. Inspecting trace plots, it is clear that the highest scoring traces are the most caterpillar-like while medium-scoring traces have more irregularities and very low-scoring traces are highly irregular with no discernible pattern (see figures 5.3, 5.4 and 5.5).

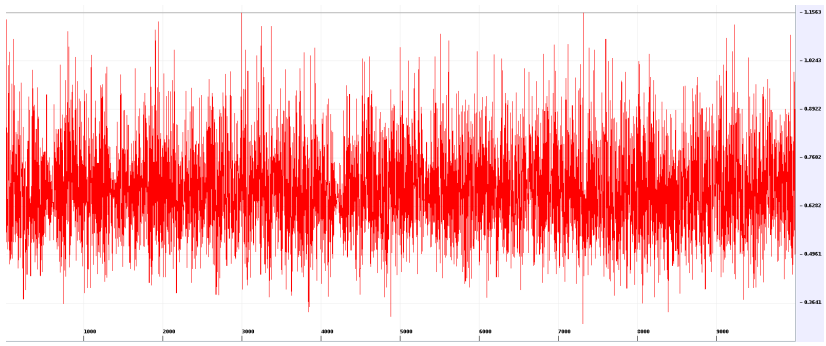

Figure 5.3: Full trace plot for the parameter *Tree_weights4.edgeWeight[9]* in the yeast trace. 10^6 iterations with sampling every 100th iteration. The highest FWSR scoring parameter with moderately good values of $\varepsilon(10^5) \approx 0.0654$ and $\widehat{ESS}(10^5) \approx 3,692$. The trace has a catterpillar shape. x-axis: number of iterations $\times$ 100. y-axis: parameter value.

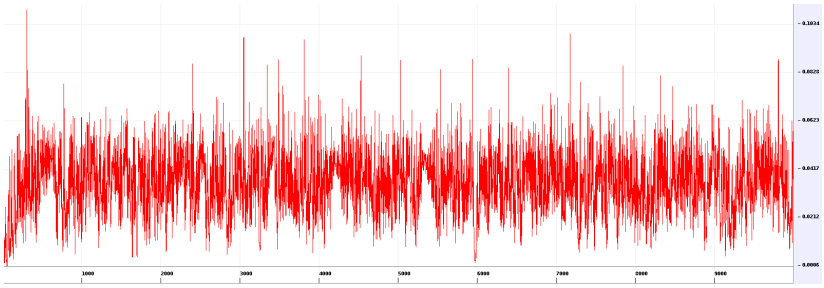

Figure 5.4: Full trace plot for the parameter *Tree_weights4.edgeWeight[19]* in the yeast trace. 10^6 iterations with sampling every 100th iteration. Moderately low FWSR scoring parameter with values $\varepsilon(10^5) \approx 0.1423$ and $\widehat{ESS}(10^5) \approx 847$. The trace has a somewhat caterpillar-like shape, with more anomalies than that seen in figure 5.3. x-axis: number of iterations $\times$ 100. y-axis: parameter value.

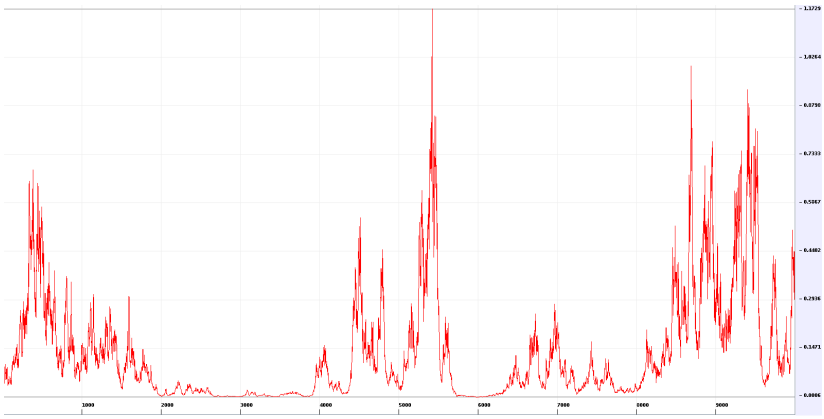

Figure 5.5: Full trace plot for the parameter *Tree_DupLoss1.deathRate* in the yeast trace. 10^6 iterations with sampling every 100th iteration. Poorly scoring parameter with values $\varepsilon(10^5) \approx 0.3537$ and $\widehat{ESS}(10^5) \approx 123$. The trace has a highly irregular shape. x-axis: number of iterations $\times$ 100. y-axis: parameter value.

5.2.4 Decreasing DS1 run length

As seen above, the DS1 traces achieved much better FWSR results than the yeast trace. Remember that the latter trace had a size of 10^4 while the former two had size 10^5. As we would expect, taking only the initial 10^4 sample points from the DS1 traces significantly decreases their FWSR scores. The previously best scoring parameter in trace 1 had $\widehat{ESS}(10^5) \approx 16,653$ but only $\widehat{ESS}(10^4) \approx 1,485$. The worst scoring parameter in that trace had $\widehat{ESS}(10^5) \approx 6,122$ but got $\widehat{ESS}(10^4) \approx 896$. So only taking the initial tenth of the trace decreased the ESS score for the former parameter

by a factor of 11 and for the latter by a factor of 7. This suggests that a trace of size 10^4 is much too small.

5.2.5 On burn-in

Sections 3.3.1 and 3.3.3 described burn-in and the motivations for attempting to discard the burn-in portion of the chain. Indeed, one of the major features of VMCMC is in fact burn-in estimation. In section 3.3.3 it was claimed that run-length criteria could be satisfied earlier if the burn-in portion of the chain was discarded, mainly since burn-in is a major contributor to a higher standard deviation. However, it is not entirely clear whether discarding burn-in will lead to earlier FWSR success, and Gong and Flegal do not mention burn-in in relation to FWSR. Remember that FWSR determines the run length to be enough when the confidence interval $w(\delta, n)$ is sufficiently small in relation to the standard deviation $\hat{\lambda}(n)$. $w(\delta, n)$ in turn is dependent upon $\hat{\sigma}(n)$ that is calculated through the average distance of batch means to the sample mean. Thus, discarding the burn-in portion will have the effect that both $\hat{\lambda}(n)$ and $\hat{\sigma}(n)$ (and thus $w(\delta, n)$) shrink in size. For burn-in to play an important role with regards to FWSR it would therefore be required that $\hat{\sigma}(n)$ shrinks *more* in size than $\hat{\lambda}(n)$. Furthermore, remember that $w_i(\delta, n)$ for a parameter i is defined as:

$$w_i(\delta, n) := 2z_{\delta/2}\frac{\hat{\sigma}_i(n)}{\sqrt{n}}$$

The division by $\sqrt{n}$ indicates that $w(\delta, n)$ favours larger samples. For example, discarding the first 10^4 sample points from a sample of 10^5 would give $\sqrt{n} = 300$ instead of $\sqrt{n} \approx 316$ at the last iteration. The value $p(n)$ (equation 3.8), that is added to $w(\delta, n)$, also grows smaller as n increases. So there are indications that discarding burn-in could possibly even have a negative effect on FWSR performance.

To test this, we again consider trace 1 featured in section 5.2.2. FWSR without burn-in determined that a sample size of 62,280 was enough, given $\epsilon = 0.062$. VMCMC's built-in Geweke diagnostic suggests a burn-in of 5,000 for some parameters and 10,000 for some. Discarding the first 5,000 sample points increases the required sample size from 62,280 to 65,025. Discarding the first 10,000 points on the other hand decreases the required sample size to 61,009. However, since portions of the samples need to be thrown away the total number of samples to be gathered are 62,280 for burn-in 0, 70,025 for burn-in 5,000 and 71,009 for burn-in 10,000, showing that discarding burn-in actually increases the required simulation effort. The effects of burn-in removal on required sample size are minimal in this case.

$\widehat{ESS}$ values at the last iteration remain practically unchanged for most

parameters when discarding 5,000 burn-in samples, with the exception of two parameters whose $\widehat{ESS}$ values increase by about 1,300 and 1,900 respectively. Removing 10,000 gives a minor decrease in many values while the rest remain largely unchanged. Two parameters have major drops in standard deviation when burn-in is discarded; from about 28 and 27 to around 6 for both burn-in lengths. One of the parameters had $\widehat{ESS}$ increased by about 1,300 for both 5,000 and 10,000 burn-in, while the other remained largely unchanged.

In conclusion, there are indications that discarding burn-in has varying effect on FWSR performance. This does not imply that considering burn-in is unnecessary, but only that it may not be very important with regard to FWSR. Further studies will have to be done on this subject before a definite conclusion can be reached.

5.2.6 Note on thinning

All of the tested MCMC traces have been obtained while using *thinning* during the simulation. That is, sampling has not been done at every iteration but rather at every kth iteration for some k ($k = 100$ in this case). While thinning is common practice, Gong and Flegal do not mention it at all. Thus, it is unclear what role it plays in relation to FWSR. Testing that is beyond the scope of this project, but it might be desirable to do so in the future.

6 Conclusions

In this project, FWSR has been implemented as an MCMC post-processing diagnostic. Utilizing FWSR in such a way is certainly possible, although it would probably be preferable to use it as it was intended, i.e. as a stopping rule for a running simulation. Still, it may be a practical tool to have in VMCMC. FWSR has relatively high demands in terms of run length and seemingly tends to give good ratings to caterpillar-like traces with good mixing and worse ratings to traces with irregular anomalies, possibly making it useful for determining mixing on high-dimensional traces where manual inspection is time-consuming. It appears as though good FWSR results do not require a low standard deviation. It remains an open question what impact discarding burn-in has on FWSR performance, but there are indications that it is quite small.

7 Recommendations for further study

The testing in section 5.2 concerned a few MCMC traces. While that was enough to give an initial idea of FWSR behavior, further testing on a larger number of traces from additional datasets will be necessary to strenghten or reject the conclusions of this report. The relation between burn-in and FWSR performance is an area that was only explored briefly and will certainly require further study. Despite VMCMC being aimed at MCMC simulations within phylogenetics, the evaluation section (5.2) of this report has a more general approach. The test cases consist of phylogenetic MCMC data, but no attempt to discuss FWSR in relation to the area of phylogenetics in particular has been made. Thus, FWSR behaviour regarding e.g. particular phylogenetic parameters will have to be a matter of future study.

References

[1] Ali, R.H.: *VMCMC: a graphical and statistical analysis tool for Markov chain Monte Carlo traces in Bayesian phylogeny – Tutorial version 1.0* (update 2015-11-06)
https://bitbucket.org/rhali/visualmcmc

[2] Ali, R.H., Bark, M., Miró, J., Muhammad S.A., Sjöstrand J, Zubair, S.M., Abbas, R.M., Arvestad, L.: VMCMC: a graphical and statistical analysis tool for Markov chain Monte Carlo traces, *BMC Bioinformatics 18*, 97 (2017)
DOI: 10.1186/s12859-017-1505-3

[3] Biesel, H., Recursive calculation of the standard deviation with increased accuracy, *Chromatographia 10(4)*, 173–175 (1977)

[4] Flegal, J.M., Gong, L.: Relative fixed-width stopping rules for Markov chain Monte Carlo simulations, *Statistica Sinica 25*, 655–675 (2015) DOI: 10.5705/ss.2013.209

[5] Gelman, A., Rubin, D.B.: Inference from iterative simulation using multiple sequences, *Stat Sci. 4*, 457–511 (1992)

[6] Geweke, J.: Evaluating the accuracy of sampling-based approaches to the calculation of posterior moments, in Bernardo, J. M., Berger, J. O., Dawid, A. P., Smith, A.F.M. (ed.), *Bayesian Statistics 4*, Oxford, UK: Clarendon Press (1992)

[7] Gong, L., Flegal, J.M.: A practical sequential stopping rule for high-dimensional Markov Chain Monte Carlo, *Journal of Computational and Graphical Statistics 25(3)*, 684–700 (2016)
DOI: 10.1080/10618600.2015.1044092

[8] Hedges, S.B., Moberg, K.D., Maxson, L.R.: Tetrapod phylogeny inferred from 18S and 28S ribosomal RNA sequences and a review of the evidence for amniote relationships, *Mol Biol Evol 7(6)*, 607–633 (1990)

[9] Höhna, S.: Burnin estimation and convergence assessment, Chapter in Licentiate thesis *Bayesian Phylogenetic Inference* Stockholm University (2011)

[10] Jones, G.L.: On the Markov chain central limit theorem, *Probability Surveys 1*, 299–320 (2004)

[11] JSON website
http://www.json.org/

[12] Sahlin, K.: *Estimating convergence of Markov Chain Monte Carlo simulations*, Master's thesis, Stockholm University (2011)

[13] VMCMC source code (last retrieved 2017-08-07)
https://bitbucket.org/rhali/visualmcmc/

[14] VMCMC with FWSR source code
https://bitbucket.org/olofheurgren/visualmcmc/branch/fwsr

[15] Åkerborg, Ö., Sennblad, B., Arvestad, L., Lagergren, J.: Simultaneous Bayesian gene tree reconstruction and reconciliation analysis, *Proceedings of the National Academy of Sciences 106(14)*, 5714–5719 (2009)
DOI: 10.1073/pnas.0806251106